Nama : Frederico Steven Kwok

NPM : 242310037

Kelas : TI-24-PA1

Mata Kuliah : Lab. Pemrograman Web

Tugas : Pertemuan 8 – Tugas dan Latihan 7

Github : https://github.com/RicoSteven120206/PW-MERN-STACK-242310037.git

Dokumentasi Lengkap Source Code Berada di Repository Github

1. Lengkapi sistem backend dengan modul manajemen pengguna (*User Management*). Modul ini akan menangani data pengguna yang nantinya akan berinteraksi dengan koleksi buku. Dengan ketentuan seperti dibawah ini:
   a. Buatlah model yang merepresentasikan table users dengan attribute seperti table users dibawah ini:

      Source Code pada model tabel users /models/User.js

```
"use strict";
const { Model } = require("sequelize");


module.exports = (sequelize, DataTypes) => {
 class User extends Model {
  /**
   * Helper method for defining associations.
   * This method is not a part of Sequelize lifecycle.
   * The `models/index` file will call this method automatically.
   */
  static associate(models) {
   // define association here
   // contoh: User.hasMany(models.Book, { foreignKey: "user_id" });
  }
 }
```

```
  User.init(
   {
    email: {
     type: DataTypes.STRING(100),
     allowNull: true,
    },
    username: {
     type: DataTypes.STRING(50),
     allowNull: true,
    },
    password: {
     type: DataTypes.STRING(255),
     allowNull: true,
    },
    is_active: {
     type: DataTypes.BOOLEAN, // tinyint(1) di MySQL dipetakan ke
BOOLEAN
     allowNull: true,
    },
   },
   {
    sequelize,
    modelName: "User",
    tableName: "users",
    timestamps: true, // otomatis kelola createdAt & updatedAt
   }
  );

  return User;
};
```

b. Jalankan migrasi untuk memastikan table users terbentuk dengan benar di database.

Untuk membuat isi file nya di migration dengan mengetik dibawah berikut, setelah itu isi migration pada source code dibawah.

```
npx sequelize-cli migration:generate --name create-books
```

Source Code untuk migration 20260611023601-create-books

```
"use strict";
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable("users", {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER,
      },
      email: {
        type: Sequelize.STRING(100),
        allowNull: true,
      },
      username: {
        type: Sequelize.STRING(50),
        allowNull: true,
      },
      password: {
        type: Sequelize.STRING(255),
        allowNull: true,
      },
      is_active: {
        type: Sequelize.BOOLEAN,
        allowNull: true,
```

```
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE,
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE,
      },
    });
  },

  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable("users");
  },
};
```

Untuk menjalankan migration nya seperti berikut.

```
npx sequelize-cli db:migrate
```

Akan nampilkan Output jika berhasil

```
PS D:\Tugas Kuliah Semester 4\Lab. Pemrograman Web\MERN-STACK\MERN-STACK\backend> npx sequelize-cli db:migrate

Sequelize CLI [Node: 24.14.1, CLI: 6.6.5, ORM: 6.37.8]

◇ injected env (8) from .env // tip: ⌘ override existing { override: true }
Loaded configuration file "config\config.js".
Using environment "production".
== 20260611023601-create-books: migrating =======
== 20260611023601-create-books: migrated (0.064s)

== 20260625022206-create-user: migrating =======
== 20260625022206-create-user: migrated (0.011s)
```

2. Buatkan routing rest api untuk user management seperti table dibawah ini:

Method	Endpoint	Controller Action	Keterangan
GET	/	getAllUsers	Mengambil daftar semua user.
GET	/:id	getUserById	Mengambil detail user berdasarkan ID unik.
POST	/	createUser	Menambah data user baru.
PUT	/:id	updateUser	Memperbarui seluruh data user.
DELETE	/:id	deleteUser	Menghapus user berdasarkan ID.

⇨ Source Code pada membangun Controller dan Routes

Source Code pada UserController.js

```
const db = require("../models");
const User = db.User;
const { Op } = db.Sequelize;
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");

// Login users
exports.loginUser = async (req, res) => {
 try {
  const { email, password } = req.body;

  if (!email || !password) {
   return res.status(400).json({
    success: false,
    message: "Email and password are required",
   });
  }

  // Find user by email
  const user = await User.findOne({ where: { email } });
  if (!user) {
   return res.status(401).json({
    success: false,
    message: "Invalid email or password",
   });
  }

  // Check if user is active
  if (!user.is_active) {
   return res.status(403).json({
    success: false,
    message: "Account is inactive. Please contact administrator",
   });
  }

  // Verify password
  const isPasswordValid = await bcrypt.compare(password, user.password);
  if (!isPasswordValid) {
   return res.status(401).json({
    success: false,
    message: "Invalid email or password",
   });
  }

  const userResponse = {
```

```
      id: user.id,
      email: user.email,
      username: user.username,
    };

    const expirationTime = Math.floor(Date.now() / 1000) + 6 * 60 * 60;
    const accessToken = jwt.sign(userResponse, process.env.JWT_SECRET, {
      expiresIn: "6h",
    });

    res.json({
      success: true,
      message: "Login successful",
      data: userResponse,
      accessToken,
      expiresIn: expirationTime,
    });
  } catch (error) {
    console.error("Error during login:", error);
    res.status(500).json({
      success: false,
      message: "Failed to login",
      error: error.message,
    });
  }
};

// Get all users
exports.getAllUsers = async (req, res) => {
  try {
    const { search, is_active, sort_by, order } = req.query;

    // Build where clause
    let whereClause = {};

    if (search) {
      whereClause = {
        ...whereClause,
        [Op.or]: [
          { username: { [Op.like]: `%${search}%` } },
          { email: { [Op.like]: `%${search}%` } },
        ],
      };
    }

    if (is_active !== undefined) {
      whereClause.is_active = is_active === "true";
```

```
    }

    // Build order clause
    let orderClause = [["created_at", "DESC"]];
    if (sort_by) {
      const sortOrder = order && order.toUpperCase() === "ASC" ? "ASC" :
"DESC";
      orderClause = [[sort_by, sortOrder]];
    }

    const users = await User.findAll({
      where: whereClause,
      order: orderClause,
      attributes: { exclude: ["password"] }, // Exclude password from response
    });

    res.json({
      success: true,
      count: users.length,
      data: users,
    });
  } catch (error) {
    console.error("Error fetching users:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch users",
      error: error.message,
    });
  }
};

// Get single user by ID
exports.getUserById = async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id, {
      attributes: { exclude: ["password"] },
    });

    if (!user) {
      return res.status(404).json({
        success: false,
        message: "User not found",
      });
    }

    res.json({
      success: true,
```

```
      data: user,
    });
  } catch (error) {
    console.error("Error fetching user:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch user",
      error: error.message,
    });
  }
};

// Create new user (Register)
exports.createUser = async (req, res) => {
  try {
    const { email, username, password, is_active } = req.body;

    // Validation
    if (!email || !username || !password) {
      return res.status(400).json({
        success: false,
        message: "Email, username, and password are required",
      });
    }

    // Check if email already exists
    const existingEmail = await User.findOne({ where: { email } });
    if (existingEmail) {
      return res.status(400).json({
        success: false,
        message: "Email already exists",
      });
    }

    // Check if username already exists
    const existingUsername = await User.findOne({ where: { username } });
    if (existingUsername) {
      return res.status(400).json({
        success: false,
        message: "Username already exists",
      });
    }

    // Hash password
    const saltRounds = 10;
    const hashedPassword = await bcrypt.hash(password, saltRounds);
```

```
    // Create user
    const user = await User.create({
      email,
      username,
      password: hashedPassword,
      is_active: is_active !== undefined ? is_active : true,
    });

    // Remove password from response
    const userResponse = user.toJSON();
    delete userResponse.password;

    res.status(201).json({
      success: true,
      message: "User created successfully",
      data: userResponse,
    });
  } catch (error) {
    console.error("Error creating user:", error);

    // Handle validation errors
    if (error.name === "SequelizeValidationError") {
      return res.status(400).json({
        success: false,
        message: "Validation error",
        errors: error.errors.map((e) => ({
          field: e.path,
          message: e.message,
        })),
      });
    }

    res.status(500).json({
      success: false,
      message: "Failed to create user",
      error: error.message,
    });
  }
};

// Update user (full update)
exports.updateUser = async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id);

    if (!user) {
      return res.status(404).json({
```

```
      success: false,
      message: "User not found",
    });
  }

  const { email, username, password, is_active } = req.body;

  // Check if email is being changed and already exists
  if (email && email !== user.email) {
    const existingEmail = await User.findOne({ where: { email } });
    if (existingEmail) {
      return res.status(400).json({
        success: false,
        message: "Email already exists",
      });
    }
  }

  // Check if username is being changed and already exists
  if (username && username !== user.username) {
    const existingUsername = await User.findOne({ where: { username } });
    if (existingUsername) {
      return res.status(400).json({
        success: false,
        message: "Username already exists",
      });
    }
  }

  // Prepare update data
  const updateData = {
    email: email || user.email,
    username: username || user.username,
    is_active: is_active !== undefined ? is_active : user.is_active,
  };

  // Hash new password if provided
  if (password) {
    const saltRounds = 10;
    updateData.password = await bcrypt.hash(password, saltRounds);
  }

  await user.update(updateData);

  // Remove password from response
  const userResponse = user.toJSON();
  delete userResponse.password;
```

```
    res.json({
      success: true,
      message: "User updated successfully",
      data: userResponse,
    });
  } catch (error) {
    console.error("Error updating user:", error);

    // Handle validation errors
    if (error.name === "SequelizeValidationError") {
      return res.status(400).json({
        success: false,
        message: "Validation error",
        errors: error.errors.map((e) => ({
          field: e.path,
          message: e.message,
        })),
      });
    }

    res.status(500).json({
      success: false,
      message: "Failed to update user",
      error: error.message,
    });
  }
};

// Patch user (partial update)
exports.patchUser = async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id);

    if (!user) {
      return res.status(404).json({
        success: false,
        message: "User not found",
      });
    }

    const { email, username, password, is_active } = req.body;

    // Check if email is being changed and already exists
    if (email && email !== user.email) {
      const existingEmail = await User.findOne({ where: { email } });
      if (existingEmail) {
```

```
        return res.status(400).json({
          success: false,
          message: "Email already exists",
        });
      }
    }

    // Check if username is being changed and already exists
    if (username && username !== user.username) {
      const existingUsername = await User.findOne({ where: { username } });
      if (existingUsername) {
        return res.status(400).json({
          success: false,
          message: "Username already exists",
        });
      }
    }

    // Prepare update data
    const updateData = { ...req.body };

    // Hash new password if provided
    if (password) {
      const saltRounds = 10;
      updateData.password = await bcrypt.hash(password, saltRounds);
    }

    await user.update(updateData);

    // Remove password from response
    const userResponse = user.toJSON();
    delete userResponse.password;

    res.json({
      success: true,
      message: "User updated successfully",
      data: userResponse,
    });
  } catch (error) {
    console.error("Error updating user:", error);

    if (error.name === "SequelizeValidationError") {
      return res.status(400).json({
        success: false,
        message: "Validation error",
        errors: error.errors.map((e) => ({
          field: e.path,
```

```
            message: e.message,
          })),
        });
      }

      res.status(500).json({
        success: false,
        message: "Failed to update user",
        error: error.message,
      });
    }
  };

  // Delete user
  exports.deleteUser = async (req, res) => {
    try {
      const user = await User.findByPk(req.params.id);

      if (!user) {
        return res.status(404).json({
          success: false,
          message: "User not found",
        });
      }

      await user.destroy();

      res.json({
        success: true,
        message: "User deleted successfully",
      });
    } catch (error) {
      console.error("Error deleting user:", error);
      res.status(500).json({
        success: false,
        message: "Failed to delete user",
        error: error.message,
      });
    }
  };
```

Source Code pada UserRoutes.js

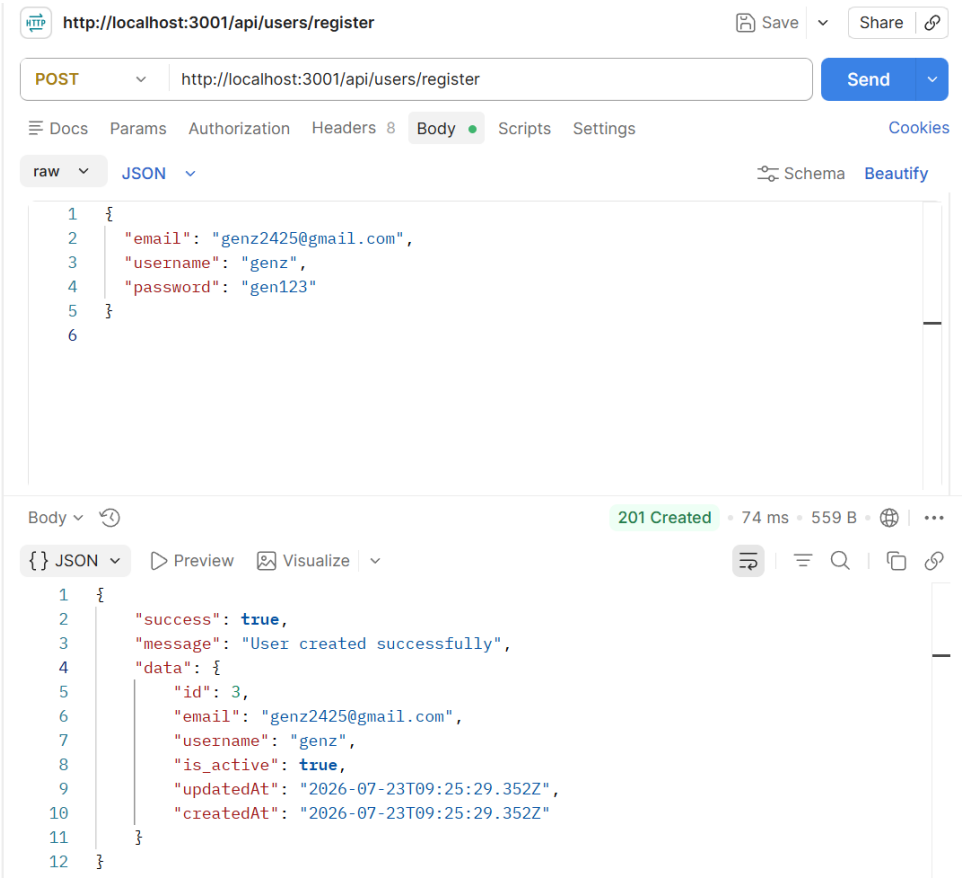
```
  const express = require("express");
  const router = express.Router();
  const userController = require("../controllers/userController");
  const { verifyToken } = require('../middleware/auth');

  router.post('/login', userController.loginUser);
```

```
router.post('/register', userController.createUser);

router.use(verifyToken);

router.get("/", userController.getAllUsers);
router.get("/:id", userController.getUserById);
router.post("/", userController.createUser);
router.put("/:id", userController.updateUser);
router.patch("/:id", userController.patchUser);
router.delete("/:id", userController.deleteUser);

module.exports = router;
```

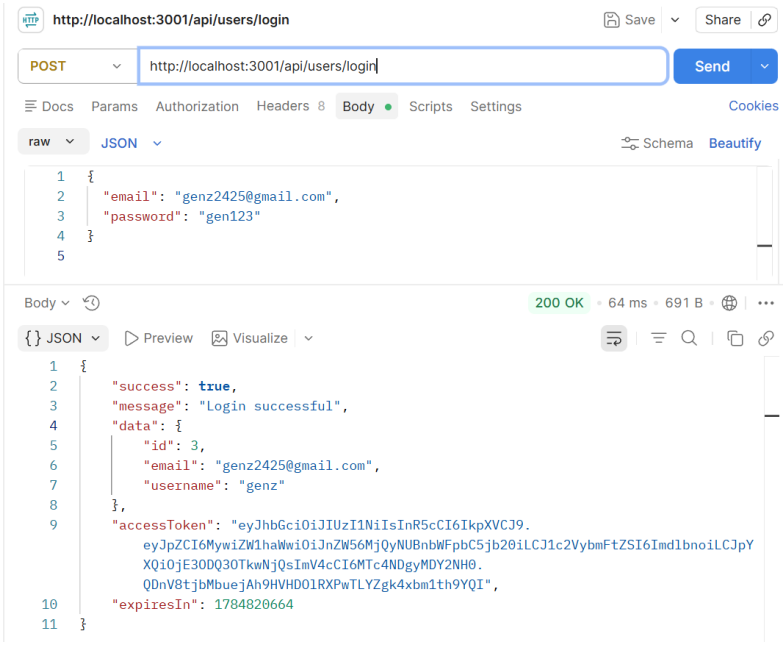
3. Lakukan pengujian menggunakan aplikasi penguji API (seperti Postman atau Insomnia) untuk memastikan bahwa setiap endpoint mengembalikan status kode HTTP yang benar (Contoh: 200 untuk sukses, 201 untuk data baru, 404 jika user tidak ditemukan).

   ⇨ Screenshot pengujian HTTP Status Code

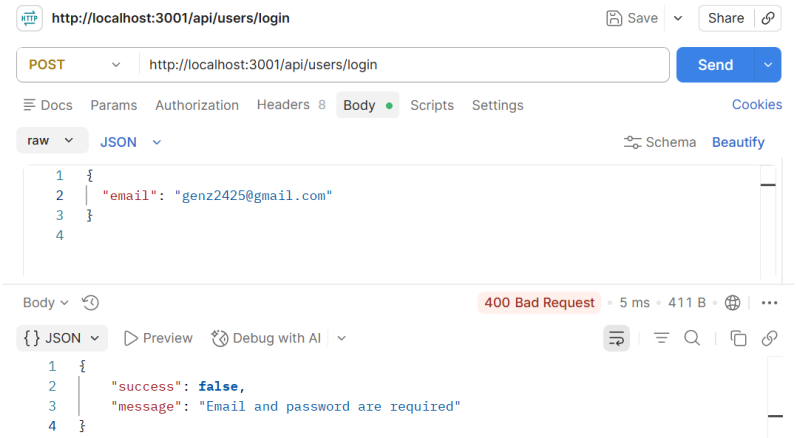
   Status Code HTTP 201 – Register => POST api/users/register



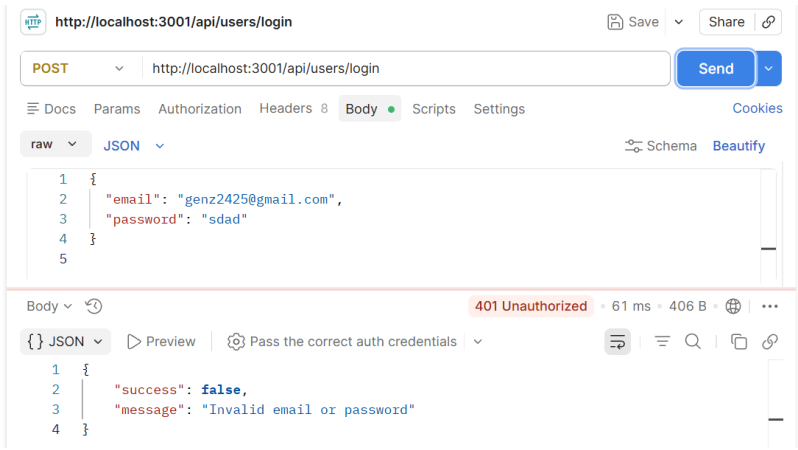
Status Code HTTP 200 – Login => POST api/users/login



Status Code HTTP 400 – Request Buruk disebabkan ada Field kosong atau tidak sesuai



Status Code HTTP 401 – Unauthorized disebabkan karena tidak sesuai terdaftar atau field tidak sama.



Status Code 403 – Forbidden disebabkan karena Token Palsu


```
"accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
    eyJpZCI6MywiZW1haWwiOiJnZW56MjQyNUBnbWFpbC5jb20iLCJ1c2VybmFtZSI6Imdlbnoi LCJpY
    XQiOjE3ODQ4MDA5MjIsImV4cCI6MTc4NDgyMjUyMn0.
    ZFq-3yZHtfmgIJiSXcxV_X9lJv0Y7VRADWEBvFCT7tU",
"expiresIn": 1784822522
```


Biasanya akan gagal untuk mendapatkan akses.

Status	Kapan Muncul atau Terjadi
200 OK	Berhasil GET/PUT/PATCH/DELETE
201 Created	Berhasil Register/Create User
400 Bad Request	Field wajib kosong, email/username duplikat
401 Unauthorized	Login gagal, atau tidak kirim token sama sekali
403 Forbidden	Token ada tapi tidak valid/rusak, atau akun nonaktif
404 Not Found	ID user tidak ditemukan

4. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran soal nomor 1-3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.

   ⇨ Done. Pengerjaan Dokumentasi Selesai.

5. Upload project tersebut pada GITHUB dengan menggunakan repository bernama "**PW-MERN-STACK-NPM**" dengan nama commit "**Praktikum 7**".

   Link GITHUB PW-MERN-STACK-242310037

   https://github.com/RicoSteven120206/PW-MERN-STACK-242310037.git